

Parallel and Distributed Computing



Assessments # 02

Submitted By

QASIM ALI (20P-0070)

Submitted to: Ali Sayyed

Section: BSCS-6A

**DEPARTMENT OF COMPUTER SCIENCE
FAST NATIONAL UNIVERSITY OF COMPUTER
AND EMERGING SCIENCES, PESHAWAR**

Task 1

a. How is workload distribution affected by the number of processes?

- With more processes, the workload is divided into smaller parts, so each process works on fewer elements.
- If the number of elements doesn't evenly divide by the number of processes (e.g., $16 \% \text{ num procs} \neq 0$), some processes will handle one extra element.
- To make sure the work is distributed evenly, the program has logic that adjusts for this uneven split by giving the first few processes one more element.

b. Can this design scale for larger arrays? Why or why not?

- This approach works fine for moderately large arrays but has its limits.
- A key problem is that the master process (rank 0) is in charge of reading input, sending data, and collecting results, which can slow things down.
- To handle much larger arrays or when there are many processes, the design would need:
 - o Non-blocking communication (like `MPI_Isend`)
 - o Parallel handling of input/output (I/O)
 - o Better load balancing among processes

Explanation of Code:

This MPI-based program is designed to square 16 numbers by splitting the task across several processes. The master process (rank 0) takes care of input, divides the work, and gathers the results. The worker processes receive their part of the data, square the numbers, and send the results back.

The program includes logic to handle cases where the number of elements doesn't divide perfectly by the number of processes. It gives the extra elements to the first few processes to ensure a balanced workload. Each process works on its portion of the data based on its rank, which makes the program flexible for different numbers of processes (up to 16).

```

C task1.c X
D: > 2025 > pdc > Assignment#02 > C task1.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  #define TOTAL_NUMBERS 16 // Total number of integers to process
6
7  int main(int argc, char *argv[]){
8
9      int rank, num_processes;
10     int data[TOTAL_NUMBERS]; // Full array of numbers (handled only by master process)
11     int result[TOTAL_NUMBERS]; // Final array to store squared numbers (filled by master)
12
13     MPI_Init(&argc, &argv); // Start the MPI environment
14     MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get the current process ID
15     MPI_Comm_size(MPI_COMM_WORLD, &num_processes); // Get the total number of processes
16
17     if(num_processes > TOTAL_NUMBERS){
18
19         if(rank == 0){
20             // Too many processes compared to the number of elements
21             printf("Error: Number of processes must be <= %d\n", TOTAL_NUMBERS);
22         }
23
24         MPI_Finalize();
25         return 0;
26     }
27
28     int base = TOTAL_NUMBERS / num_processes; // Base number of elements each process should handle
29     int extra = TOTAL_NUMBERS % num_processes; // Remaining elements to be distributed one-by-one to first few processes
30
31     // Determine how many elements this process will handle
32     int local_count;
33     if(rank < extra){
34         // First 'extra' processes get one additional element
35         local_count = base + 1;

```

```

    int local_count;
    if(rank < extra){
        // First 'extra' processes get one additional element
        local_count = base + 1;
    }
    else{
        local_count = base;
    }

    // Compute the starting index for this process's data in the global array
    int start_index = 0;
    for(int i = 0; i < rank; i++){

        if(i < extra){
            start_index += base + 1; // Account for extra elements in earlier processes
        }
        else{
            start_index += base;
        }
    }

    int local_data[16]; // Buffer to hold this process's chunk of data
    int local_result[16]; // Buffer to hold squared results for this process

    // Master process will read the input and distribute chunks to all processes
    if(rank == 0){

        printf("Enter %d numbers:\n", TOTAL_NUMBERS);
        for(int i = 0; i < TOTAL_NUMBERS; i++){
            scanf("%d", &data[i]); // Input the numbers from user
        }
    }

```

```

// Distribute chunks of data to each worker process
for(int i = 1; i < num_processes; i++){

    int send_count;
    if(i < extra){
        send_count = base + 1; // Give extra element to early processes
    }
    else{
        send_count = base;
    }

    // Calculate where this chunk begins in the data array
    int send_index = 0;
    for(int j = 0; j < i; j++){

        if(j < extra){
            send_index += base + 1;
        }
        else{
            send_index += base;
        }
    }

    // Send the appropriate chunk to the i-th process
    MPI_Send(&data[send_index], send_count, MPI_INT, i, 0, MPI_COMM_WORLD);
}

// Master computes squares of its own chunk
for(int i = 0; i < local_count; i++){
    local_result[i] = data[i] * data[i];
}

```

```

// Copy master's results into the result array
for(int i = 0; i < local_count; i++){
    result[i] = local_result[i];
}

// Collect the squared results from all worker processes
int offset = local_count;
for(int i = 1; i < num_processes; i++){

    int rcv_count;
    if(i < extra){
        rcv_count = base + 1;
    }
    else{
        rcv_count = base;
    }

    // Receive results into the correct position in result array
    MPI_Recv(&result[offset], rcv_count, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    offset += rcv_count;
}

// Display the final squared result array
printf("Final squared array:\n");
for(int i = 0; i < TOTAL_NUMBERS; i++){
    printf("%d ", result[i]);
}
printf("\n");
}

```

```

// Worker processes: receive their chunk, square each number, and send result back
else{

    MPI_Recv(local_data, local_count, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    for(int i = 0; i < local_count; i++){
        local_result[i] = local_data[i] * local_data[i]; // Square each element
    }

    MPI_Send(local_result, local_count, MPI_INT, 0, 0, MPI_COMM_WORLD); // Send back to master
}

MPI_Finalize(); // Shut down MPI
return 0;
}

```

Output:

```

~/mnt/d/OpenMPI$ mpicc -o hello task1.c
~/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Enter 16 numbers:
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Final squared array:
25 36 49 64 81 100 121 144 169 196 225 256 289 324 361 400

```

Task 2

a. Why is MPI_Waitall needed?

- MPI_Waitall ensures that all non-blocking operations, like MPI_Isend and MPI_Irecv, are fully completed before moving on. Without it, the program might continue without waiting for the communication to finish, which can cause issues with data transfer.

b. What happens if you omit waiting (like removing MPI_Waitall)?

- If you remove MPI_Waitall, the program may:
 - o Print incorrect or garbage data
 - o Miss values during communication
 - o Hang or crash unexpectedly
- This happens because the communication may still be ongoing when the program tries to use the data, leading to incomplete or inconsistent results.
- You can test this by commenting out MPI_Waitall(...) and running the program again—you'll likely get incorrect or incomplete output.

Explanation of Code:

This MPI program takes 16 numbers from the user, splits the task across multiple processes, and squares the numbers using non-blocking communication (MPI_Isend and MPI_Irecv). The master process (rank 0) handles the input, processes its own portion, and sends the remaining data to worker processes. It also sets up non-blocking sends and receives, allowing communication to occur without waiting for each operation to finish immediately.

Each worker process receives its part of the data, squares the numbers, and sends the results back to the master using non-blocking communication. The master process waits for all communication to finish using MPI_Waitall before it prints the final results.

The key feature of this version is its efficiency: by not waiting after every send/receive, the processes can perform calculations while data is being transferred, which helps improve overall performance, especially in larger systems.

```
D:\> 2025 > pdc > Assignment#02 > C task2.c > main(int, char* [])
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  // Macro to define the total number of input elements
6  #define TOTAL_NUMBERS 16
7
8  int main(int argc, char *argv[])
9
10     int rank, num_processes;
11     int data[TOTAL_NUMBERS]; // Only used by master to store input
12     int result[TOTAL_NUMBERS]; // Only used by master to store output
13
14     // Initialize MPI environment
15     MPI_Init(&argc, &argv);
16
17     // Get the rank of the current process
18     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
19
20     // Get the total number of processes
21     MPI_Comm_size(MPI_COMM_WORLD, &num_processes);
22
23     // Make sure the number of processes does not exceed data elements
24     if(num_processes > TOTAL_NUMBERS){
25
26         if(rank == 0){
27             // Only the master prints this error
28             printf("Only up to %d processes are allowed.\n", TOTAL_NUMBERS);
29         }
30
31         MPI_Finalize();
32         return 0;
33     }
34
35     int base = TOTAL_NUMBERS / num_processes; // Minimum number of elements per process
36     int extra = TOTAL_NUMBERS % num_processes; // Remainder elements to distribute
37
38     // Master process logic
39     if(rank == 0){
40
41         printf("Enter %d numbers:\n", TOTAL_NUMBERS);
42
43         // Read all numbers from user input
44         for(int i = 0; i < TOTAL_NUMBERS; i++){
45             scanf("%d", &data[i]);
46         }
47
48         // Arrays to manage non-blocking communication
49         MPI_Request send_requests[num_processes - 1];
50         MPI_Request recv_requests[num_processes - 1];
51         MPI_Status statuses[num_processes - 1];
52
53         int current_index = 0; // Index for traversing input array
54         int result_index = 0; // Index for filling result array
55     }
```

```

// Master determines how many elements it will process
if(extra > 0){
    local_count = base + 1;
}
else{
    local_count = base;
}

// Master squares its portion of the array
for(int i = 0; i < local_count; i++){
    result[result_index] = data[current_index] * data[current_index];
    current_index++;
    result_index++;
}

int req_index = 0;

// Send chunks to worker processes using non-blocking sends/receives
for(int i = 1; i < num_processes; i++){

    int send_count;

    if(i < extra){
        send_count = base + 1; // Give extra elements to early processes
    }
    else{
        send_count = base;
    }

    // Start non-blocking send and receive for each worker
    MPI_Isend(&data[current_index], send_count, MPI_INT, i, 0, MPI_COMM_WORLD, &send_requests[req_index]);
    MPI_Irecv(&result[result_index], send_count, MPI_INT, i, 0, MPI_COMM_WORLD, &recv_requests[req_index]);

    current_index += send_count;
    result_index += send_count;
    req_index++;
}

// Wait for all sends and receives to complete
if(num_processes > 1){
    MPI_Waitall(num_processes - 1, send_requests, statuses);
    MPI_Waitall(num_processes - 1, recv_requests, statuses);
}

// Display final results
printf("Final squared array:\n");

for(int i = 0; i < TOTAL_NUMBERS; i++){
    printf("%d ", result[i]);
}

```

```

// Worker process logic
else{

    int local_count;

    // Determine how many elements this process will handle
    if(rank < extra){
        local_count = base + 1;
    }
    else{
        local_count = base;
    }

    int recv_data[local_count];    // Buffer to receive data from master
    int squared_data[local_count]; // Buffer to store squared results

    MPI_Request recv_req;
    MPI_Request send_req;

    // Receive chunk of data from master (non-blocking)
    MPI_Irecv(recv_data, local_count, MPI_INT, 0, 0, MPI_COMM_WORLD, &recv_req);
    MPI_Wait(&recv_req, MPI_STATUS_IGNORE); // Wait for data to arrive

    // Square each received number
    for(int i = 0; i < local_count; i++){
        squared_data[i] = recv_data[i] * recv_data[i];
    }

    // Send back the results to master (non-blocking)
    MPI_Isend(squared_data, local_count, MPI_INT, 0, 0, MPI_COMM_WORLD, &send_req);
    MPI_Wait(&send_req, MPI_STATUS_IGNORE); // Wait for send to complete
}

MPI_Finalize(); // Clean up the MPI environment
return 0;
}

```

Output:

```

~/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Enter 16 numbers:
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Final squared array:
25 36 49 64 81 100 121 144 169 196 225 256 289 324 361 400

```

Task 3

a. How do message tags help when many messages are sent at once?

Tags act like name labels attached to each message. They help the receiving process figure out what each message is for—even if multiple messages come from different sources at the same time. This keeps the communication organized and avoids confusion.

b. What if two messages with the same tag come from different sources?

If both messages have the same tag and the receiver uses `MPI_ANY_SOURCE`, it might grab them in the wrong order. This can cause the program to mix up the messages, use the wrong data, or give incorrect results—especially when each message was meant for a different purpose.

Explanation of Code:

This MPI program shows how processes can talk to each other in a more controlled way using message tags. It uses four processes to do different parts of the job:

- Process 0 (the main one) sends two different arrays—one goes to Process 1 and the other to Process 2.
- Process 1 takes its array and squares each number, then sends it to Process 3.
- Process 2 takes its array and doubles each number, then also sends it to Process 3.
- Process 3 waits to receive both results. It uses tags to tell which result came from which process, then adds the values from both arrays together and prints the final result.

Code:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  #define SIZE 4 // Macro for the size of the arrays
6
7  int main(int argc, char *argv[]){
8
9      int rank;
10
11      MPI_Init(&argc, &argv); // Initialize MPI environment
12      MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get the rank of the current process
13
14      int array1[SIZE] = {1, 2, 3, 4}; // Array to be processed by Process 1
15      int array2[SIZE] = {5, 6, 7, 8}; // Array to be processed by Process 2
16      int result1[SIZE]; // Will hold squared values from Process 1
17      int result2[SIZE]; // Will hold doubled values from Process 2
18
19      if(rank == 0){
20
21          // Master process sends array1 to Process 1 (tag 10)
22          MPI_Send(array1, SIZE, MPI_INT, 1, 10, MPI_COMM_WORLD);
23
24          // Master process sends array2 to Process 2 (tag 20)
25          MPI_Send(array2, SIZE, MPI_INT, 2, 20, MPI_COMM_WORLD);
26      }
27      else if(rank == 1){
28
29          int rcv_arr[SIZE];
30          MPI_Status status;
31
32          // Process 1 receives array1 from Process 0 (tag 10)
33          MPI_Recv(rcv_arr, SIZE, MPI_INT, 0, 10, MPI_COMM_WORLD, &status);
34
35          // Process 1 squares each element
36          for(int i = 0; i < SIZE; i++){
37              rcv_arr[i] = rcv_arr[i] * rcv_arr[i];
38          }
39
40          // Sends squared result to Process 3 (tag 100)
41          MPI_Send(rcv_arr, SIZE, MPI_INT, 3, 100, MPI_COMM_WORLD);
42      }
43      else if(rank == 2){
44
45          int rcv_arr[SIZE];
46          MPI_Status status;
47
48          // Process 2 receives array2 from Process 0 (tag 20)
49          MPI_Recv(rcv_arr, SIZE, MPI_INT, 0, 20, MPI_COMM_WORLD, &status);
50
51          // Process 2 doubles each element
52          for(int i = 0; i < SIZE; i++){
53              rcv_arr[i] = rcv_arr[i] * 2;
54          }

```

```

    // Sends doubled result to Process 3 (tag 200)
    MPI_Send(recv_arr, SIZE, MPI_INT, 3, 200, MPI_COMM_WORLD);
}
else if(rank == 3){

    MPI_Status status;
    int recv_arr[SIZE];

    // Process 3 will receive results from both Process 1 and 2
    for(int i = 0; i < 2; i++){

        // Receives from any source and distinguishes by tag
        MPI_Recv(recv_arr, SIZE, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        // If tag is 100, it's from Process 1 (squared data)
        if(status.MPI_TAG == 100){
            for(int j = 0; j < SIZE; j++){
                result1[j] = recv_arr[j];
            }
        }
        // If tag is 200, it's from Process 2 (doubled data)
        else if(status.MPI_TAG == 200){
            for(int j = 0; j < SIZE; j++){
                result2[j] = recv_arr[j];
            }
        }
    }

    // Print the final combined result (squared + doubled values)
    printf("Final Aggregated Result:\n");

    for(int i = 0; i < SIZE; i++){

        int final;
        final = result1[i] + result2[i];
        printf("%d ", final);
    }

    printf("\n");
}

MPI_Finalize();
return 0;
}

```

Output:

```

-110LLPP:/mnt/d/OpenMPI$ mpicc -o hello task3.c
-110LLPP:/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Final Aggregated Result:
11 16 23 32

```

Task 4

a. What problems can happen in ring-based communication?

- **Deadlock:** This happens when a process waits for a message that never comes. For example, if one process forgets to send the counter, the whole system might freeze.
- **Lost messages:** If a message isn't sent or passed properly to the next process, the ring breaks, and communication fails.

- Bad timing: If processes don't send and receive at the right time, the counter might be wrong or the program might end too soon.

b. How would it change if we used two-way (bi-directional) communication?

- Each process would need to talk to both neighbours—left and right—instead of just one.
- That means it would have to keep track of two incoming messages and two outgoing ones.
- This would make things more complicated and require better timing and coordination between all the processes.

Explanation of Code:

This program uses MPI to create a simple ring of 4 processes. A counter is passed around the ring 5 times.

- The first process (rank 0) starts the counter at 0 and sends it to the next process.
- Each process gets the counter from the previous one, adds 1, and sends it forward to the next process in the ring.
- After the counter finishes going around the ring 5 times, the first process sends a stop message to the rest so they can exit safely.
- Every process prints out what it received and when it is done.

This is a good way to show how circular message passing works in MPI. It's similar to how a token ring network or some distributed algorithms pass messages between connected units.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  #define N 4 // Number of processes in the ring
6  #define M 5 // Number of complete cycles
7
8  int main(int argc, char *argv[]){
9
10     int rank;
11     int num_processes;
12     int counter;
13     int terminate_flag;
14     MPI_Status status;
15
16     counter = 0;
17     terminate_flag = 0;
18
19     MPI_Init(&argc, &argv); // Initialize MPI environment
20     MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get the rank of the current process
21     MPI_Comm_size(MPI_COMM_WORLD, &num_processes); // Get total number of processes
22
23     // Check if enough processes are available to form a ring
24     if(num_processes < N){
25
26         if(rank == 0){
27             printf("At least %d processes are required.\n", N);
28         }
29
30         MPI_Finalize();
31         return 0;
32     }
33
34     // Logic for Process 0 (initiator and controller)
35     if(rank == 0){
36
37         printf("Process %d: Starting the counter at 0.\n", rank);
38
39         // Start the ring by sending the initial counter to Process 1
40         MPI_Send(&counter, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
41
42         // Loop to complete M cycles of counter passing
43         for(int cycle = 0; cycle < M; cycle++){
44
45             // Receive counter from the last process in the ring
46             MPI_Recv(&counter, 1, MPI_INT, N - 1, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
47
48             printf("Process %d: Received counter = %d from Process %d\n", rank, counter, status.MPI_SOURCE);
49
50             // Increment the counter and send to the next process
51             counter = counter + 1;
52             MPI_Send(&counter, 1, MPI_INT, (rank + 1) % N, 0, MPI_COMM_WORLD);
53         }

```

```

}

// After M cycles, tell all other processes to terminate
terminate_flag = 1;

for(int i = 1; i < N; i++){
    MPI_Send(&terminate_flag, 1, MPI_INT, i, 1, MPI_COMM_WORLD);
}

printf("Process %d: Terminating after %d cycles.\n", rank, M);
}

// Logic for all other processes in the ring
else{

    // Loop to receive and forward the counter during M cycles
    for(int cycle = 0; cycle < M; cycle++){

        // Receive counter from previous process in the ring
        MPI_Recv(&counter, 1, MPI_INT, (rank - 1 + N) % N, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        printf("Process %d: Received counter = %d from Process %d\n", rank, counter, status.MPI_SOURCE);

        // Increment and pass the counter to the next process
        counter = counter + 1;
        MPI_Send(&counter, 1, MPI_INT, (rank + 1) % N, 0, MPI_COMM_WORLD);
    }

    // After M cycles, wait for termination signal from Process 0
    MPI_Recv(&terminate_flag, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);

    if(terminate_flag == 1){
        printf("Process %d: Terminating.\n", rank);
    }
}

MPI_Finalize(); // Finalize MPI environment
return 0;
}

```

Output:

```

faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpicc -o hello task4.c
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Process 0: Starting the counter at 0.
Process 1: Received counter = 0 from Process 0
Process 2: Received counter = 1 from Process 1
Process 3: Received counter = 2 from Process 2
Process 3: Received counter = 6 from Process 2
Process 3: Received counter = 10 from Process 2
Process 3: Received counter = 14 from Process 2
Process 3: Received counter = 18 from Process 2
Process 3: Terminating.
Process 2: Received counter = 5 from Process 1
Process 2: Received counter = 9 from Process 1
Process 2: Received counter = 13 from Process 1
Process 2: Received counter = 17 from Process 1
Process 2: Terminating.
Process 0: Received counter = 3 from Process 3
Process 0: Received counter = 7 from Process 3
Process 0: Received counter = 11 from Process 3
Process 0: Received counter = 15 from Process 3
Process 0: Received counter = 19 from Process 3
Process 0: Terminating after 5 cycles.
Process 1: Received counter = 4 from Process 0
Process 1: Received counter = 8 from Process 0
Process 1: Received counter = 12 from Process 0
Process 1: Received counter = 16 from Process 0
Process 1: Terminating.
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$

```

Task 5

Time taken for blocking VS Non-Blocking Communication / Output:

```

faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpicc -o hello task5A.c
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Enter 16 elements:
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Squared array:
25 36 49 64 81 100 121 144 169 196 225 256 289 324 361 400
Blocking time: 0.000065 seconds
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpicc -o hello task5B.c
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$ mpirun -np 4 ./hello
Enter 16 elements:
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Squared array:
25 36 49 64 81 100 121 144 169 196 225 256 289 324 361 400
Non-blocking time: 0.000027 seconds
faileddev@DESKTOP-11OLLPP:/mnt/d/OpenMPI$

```

As we can see in the screenshot:

- Blocking time: 0.000065 seconds
- Non-blocking time: 0.000027 seconds
- The difference is 0.000038 seconds, and this time non-blocking communication is faster.

Even though the difference is very small, this shows that non-blocking can be more efficient when used properly — it allows processes to work while communication is still happening.

About MPI_Barrier (Sync Overhead):

- MPI_Barrier makes sure all processes reach the same point before starting or ending time measurement.
- On small systems (like in this example), the delay is very tiny.
- But on bigger systems, this waiting time can grow and affect overall speed.
- Still, using barriers is important to get fair and accurate timing, especially when comparing blocking vs non-blocking methods.

So in this test, non-blocking communication wins, but in real-world cases, the results may vary depending on system size, timing, and load.

I've done some modifications in the same code as in task 1 and task 2 the main changes were simple they are as follows

Changes in Part A:

```
32
33     MPI_Barrier(MPI_COMM_WORLD); // Sync all processes before starting the timer
34     double start = MPI_Wtime(); // Start the timer
35
64
65     MPI_Barrier(MPI_COMM_WORLD); // Sync before stopping the timer
66     double end = MPI_Wtime(); // Stop the timer
67
73
74     // Display elapsed time for blocking communication and computation
75     printf("Blocking time: %f seconds\n", end - start);
```

Changes in Part B:

```
39  
40     MPI_Barrier(MPI_COMM_WORLD); // Synchronize all before timing  
41     double start = MPI_Wtime();   // Start timer  
42
```

```
59  
60     MPI_Barrier(MPI_COMM_WORLD); // Synchronize again before ending the timer  
61     double end = MPI_Wtime();     // Stop timer  
62
```

```
69  
70     // Print total time taken for the non-blocking communication and computation  
71     printf("Non-blocking time: %f seconds\n", end - start);
```